Pipelining, Hazards, Branch Prediction & gem5 Models

100 Points Possible

10/8/2026

Attempt 1

In Progress

NEXT UP: Submit Assignment

Add Comment

Unlimited Attempts Allowed

9/24/2026 to 10/8/2026

Details

EEL 5764: Computer Architecture — Fall 2026

Homework 2: Pipelining, Hazards, Branch Prediction & gem5 Models

Assigned: Thursday, Sept 24, 2026 | Due Date: Thursday, Oct 8, 2026 by 11:59 PM
Total Points: 100 Points | Submission: Canvas Upload in PDF or Word Format (.pdf or .docx)

Part A: Written Analytical Problems (75 Points)

Problem 1: 5-Stage Pipeline RAW Hazards & Hardware Forwarding [25 Points]

Consider a classical 5-stage RISC-V pipeline (IF, ID, EX, MEM, WB) executing the following instruction sequence:

```
add x1, x2, x3   # Computes x1 in EX (CC3), writes back in CC5
sub x4, x1, x5   # Reads x1 in ID
and x6, x1, x7   # Reads x1 in ID
or  x8, x1, x9   # Reads x1 in ID
```

Part A [8 Points]: Assume a baseline pipeline WITHOUT hardware forwarding logic, where register writes occur in the second half of the WB stage and register reads occur in the first half of the ID stage. How many stall cycles (or NOPs) must be inserted before each dependent instruction? Draw or describe the space-time pipeline execution grid.

Part B [9 Points]: Assume full hardware forwarding logic is added (EX/MEM -> EX and MEM/WB -> EX). How many stall cycles are required for this sequence? Identify the exact pipeline register sources and ALU inputs used for each bypass pathway.

Part C [8 Points]: Suppose the first instruction is replaced with a load instruction: 'lw x1, 0(x2)'. Why does hardware forwarding fail to eliminate all stalls in this scenario? How many stall cycles (load-use stalls) are required, and why?

Problem 2: Control Hazards & Dynamic Branch Prediction [25 Points]

A 5-stage pipeline has an ideal CPI of 1.0 when no hazard stalls occur. A benchmark workload has the following instruction mix:

- 60% Integer Arithmetic / Logic (0 stall cycles)
- 20% Load Instructions (20% of loads are followed immediately by a dependent instruction requiring 1 stall cycle)
- 20% Conditional Branch Instructions (Branch condition evaluated in EX stage; taken branches incur a 2-cycle flush penalty)

Part A [9 Points]: Calculate the Effective CPI assuming a static 'Predict Not-Taken' branch predictor, where 70% of all conditional branches in the workload are actually TAKEN.

Part B [9 Points]: Calculate the Effective CPI if the processor upgrades to a 2-bit dynamic branch history predictor that achieves a 90% prediction accuracy on branches.

Part C [7 Points]: Calculate the overall system performance speedup achieved by upgrading from the static predictor (Part A) to the 2-bit dynamic predictor (Part B).

Problem 3: Critical Path Timing & Stage Imbalance [25 Points]

The combinational logic delays for the 5 pipeline stages are: IF = 160 ps, ID = 120 ps, EX = 190 ps, MEM = 200 ps, WB = 110 ps. The pipeline register delay (setup time + clock-to-q delay) is t_reg = 20 ps.

Part A [8 Points]: Determine the minimum clock period (T_clock) and maximum operating clock frequency (f_max) of this 5-stage pipeline.

Part B [9 Points]: Compare this pipelined clock frequency against an unpipelined single-cycle implementation where T_clock = sum(stage_delays) = 780 ps. Assuming an ideal CPI of 1.0, what is the ideal speedup of the 5-stage pipeline over single-cycle execution?

Part C [8 Points]: Explain why the ideal speedup is less than the theoretical 5x limit. Identify the primary hardware microarchitectural sources of this speedup degradation.

Part B: Hands-On gem5 Exercise on UF ECE Server (25 Points)

Objective: Evaluate in-order (TimingSimpleCPU) vs. out-of-order (O3CPU) execution models on the UF ECE server (ece-gem5.ece.ufl.edu). Submissions must be submitted on Canvas in PDF or Word format (.pdf or .docx) containing terminal screenshots, the telemetry table below, a brief reflection, AND a companion .zip file containing your run scripts.

Step 1: Simulation Launches on UF ECE Server [10 Points]

Execute the pre-compiled RISC-V 'hello' binary under Syscall Emulation mode using the two command lines below on the UF ECE server:
Run 1 (In-Order):
build/RISCV/gem5.opt configs/deprecated/example/se.py --cpu-type=TimingSimpleCPU --caches -c tests/test-progs/hello/bin/riscv/linux/hello

Run 2 (Out-of-Order):
build/RISCV/gem5.opt configs/deprecated/example/se.py --cpu-type=O3CPU --caches -c tests/test-progs/hello/bin/riscv/linux/hello

Step 2: Hardware Telemetry Extraction Table [10 Points].

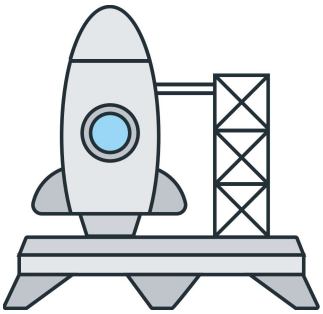
gem5 Stat Key (m5out/stats.txt)	TimingSimpleCPU (In-Order)	O3CPU (Out-of-Order)
sim_ticks	[Student Entry]	[Student Entry]
system.cpu.numCycles	[Student Entry]	[Student Entry]
system.cpu.ipc (or cpi)	[Student Entry]	[Student Entry]
system.cpu.dcache.overall_misses	[Student Entry]	[Student Entry]

Step 3: Reflection & Submission Bundle [5 Points]

Provide a 2–3 sentence reflection explaining why O3CPU achieves higher IPC/lower CPI than TimingSimpleCPU on the exact same binary. Submit your PDF or Word document on Canvas containing terminal screenshots and this table, AND upload a companion .zip file containing your gem5 execution scripts.

Choose a submission type

Upload | Google Drive | More



Choose a file to upload

File permitted: DOC, DOCX, PDF

or

Canvas Files

Submit Assignment